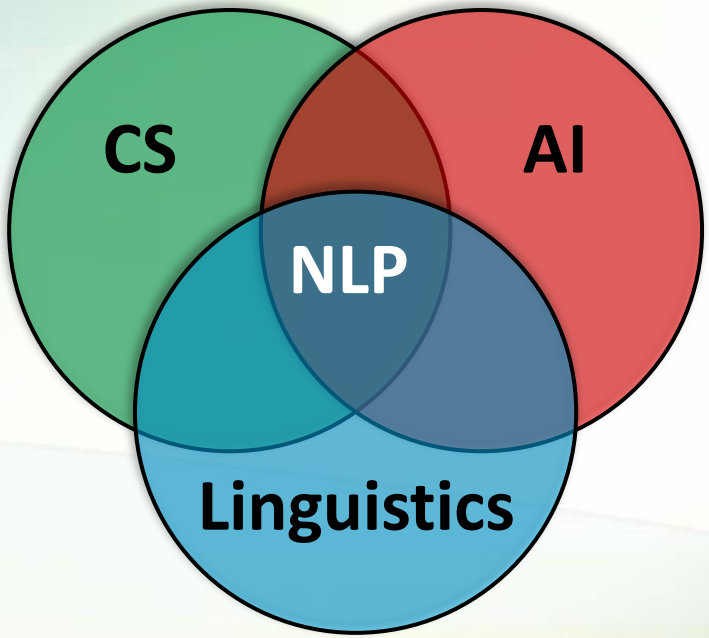
Natural Language Processing (Before LLMs)

What is NLP?



Natural Language Processing (NLP) is a field at the intersection of computer science, artificial intelligence, and linguistics that focuses on enabling computers to understand, interpret, generate, and respond to human language in a meaningful way.

Three reasons for computers to do NLP:

Communicate with humans.

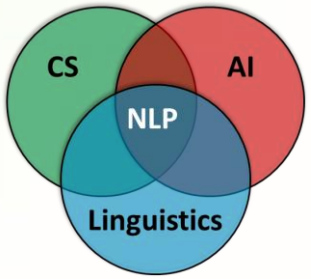
It is more convenient for humans to communicate with machines using natural language rather than a formal language like a first-order predicate calculus.

To learn.

Knowledge is stored as natural language and not in a formal logic.

Advance scientific understanding of languages and language use.

Goals / Challenge



Goals

Understanding: Extract meaning from text

Generation: Produce coherent, meaningful text

Translation: Convert between languages

Question Answering: Retrieve information from text

Dialogue: Interact naturally with humans

Challenge

Natural language is ambiguous, context-dependent, and highly variable

The chicken is ready to eat.

Ambiguous

one expression → many meanings

Is the chicken hungry, or is it cooked and ready to be eaten?

Context-Dependent

one expression; meanings depend on the situation

In a restaurant (food), on a farm (animal behavior)

Variable

Multiple expressions → one meaning

"The cooked chicken is ready to be consumed"
"The chicken is prepared"
"The chicken is done"

World Knowledge

Understanding language often requires knowledge about how the world works, not just linguistic knowledge.

Pronoun Resolution

"The trophy didn't fit in the suitcase because it was too small"	"The trophy didn't fit in the suitcase because **it** was too big"	We need to understand that objects must be smaller than their containers

Physical Constraints

"I poured water from the bottle into the cup until **it** was full"	"I poured water from the bottle into the cup until **it** was empty"	We need to understand that liquids flow from source to destination

Social/Biological Knowledge

"Bob and Joe are fathers"	"Bob and Joe are brothers"	Same grammatical structure, but different relationship implications

Temporal/Causal Reasoning

"The ice melted because it was hot"	"The ice melted so I was hot"	We need to understand "because" indicates cause, "so" indicates consequence

Historical Approaches to NLP

1950s-1980s
Symbolic/Rule-Based

- Hand-crafted grammars and rules
- Parse trees, formal grammars, logic-based reasoning
- **Limitation:** Limited coverage, brittle (breaks on unexpected input) Example
- Required enormous manual effort to handle linguistic variation

1990s-2010s
Statistical/Probabilistic

- Learn patterns from data rather than hand-code rules
- Use probability theory to handle uncertainty and variation
- N-gram models, Hidden Markov Models (HMMs), Probabilistic Context-Free Grammars (PCFGs)
- **Advantage**: Robust, handles unseen cases better

Focus of these slides.

2010s-Present
Neural Networks

- Deep learning: learn multiple levels of representation automatically
- Word embeddings (2013), Recurrent Neural Networks, Transformers (2017)
- Modern Large Language Models: GPT, BERT, Claude, ChatGPT
- **Breakthrough**: Can learn complex patterns from massive amounts of text
- Requires large datasets and computational resources

Language Models: Motivation

1990s-2010s
Statistical/Probabilistic

Complete the phrase:

Take me out to the …	ballgame
In due …	time
I forgot my …	No clear successor word.

Problem How do we assign probability to sentences (or subsequent words)?

Many NLP tasks require assessing how "likely" or "fluent" a sentence is:

Speech recognition: "I saw a van" versus "Eyes awe of an"
Homophones.

Machine translation: "strong tea" versus "powerful tea"
Both are possible, but which is more natural?

Spelling correction: "I ate dessert" versus "I ate desert"

Text generation: What word should come next?

Naïve Language Model

Definition. A **language model** is a function that assigns probabilities to finite sequences of words for a vocabulary V. For a sequence $W = w_1, w_2, \dots w_n$ where each $w_i \in V$:

$$P(W) = P(w_1, w_2, \dots w_n) \in [0,1].$$

The probabilities of all finite sequences must sum consistently with probability theory as follows:

$$\sum_{n=0}^{\infty} \sum_{w_1, w_2, \dots w_n \in V^*} P(w_1, w_2, \dots w_n) = 1$$

Examples

$P(\text{"I"}, \text{"love"}, \text{"dogs"}) = 0.00015$

$P(\text{"dogs"}, \text{"love"}, \text{"I"}) = 0.000001$ (different sequence, different probability)

$P(\text{"the"}, \text{"the"}, \text{"the"}) = 0.0$ (ungrammatical, zero probability)

Observe many sequences may have $P(W) = 0$, creating practical challenges for NLP systems.

High $P(W)$: W is a fluent, likely sentence
Low $P(W)$: W is unusual or ungrammatical

Reconsidering the Language Model Definition

Our original definition for a language model used a joint probability computation:

$$P(w_1, w_2, \dots w_n)$$

(Joint probability refers to the likelihood that two or more events occur simultaneously.)

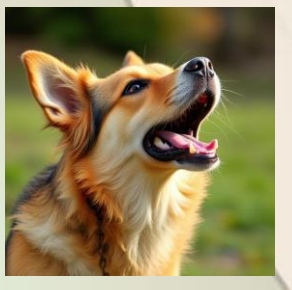
How many possible sequences?

Infinite considering *all* possible lengths!

For $n \leq 5$ and vocabulary $|V| = 20000$:

$$\sum_{n=1}^{5} |V|^n = |V|^n + \cdots + |V|^5 \approx 3.2 \cdot 10^{21}$$

We cannot possibly enumerate or store all probabilities.



How do we even estimate $P(\text{"the"}, \text{"dog"}, \text{"barked"})$?

Instead, why not ask the question:

"What is the probability of word w_i given all previous words $(w_0, \dots, w_{i-1})$?"

Language Model Version II

Any joint probability can be decomposed (using the Chain Rule of Probability):

$$P(w_1, w_2, \dots, w_n) = P(w_1) \cdot P(\, w_2 \mid w_1 \,) \cdot P(\, w_3 \mid w_1, w_2 \,) \cdot \dots \cdot P(w_n \mid w_1, \dots, w_{n-1})$$

Generally:

$$P(w_1^n) = \prod_{i=1}^{n} P(w_i \mid w_1^{i-1})$$

Definition. A **language model** is a function that assigns probabilities to words conditioned on their context (history). For a sequence $w_1, w_2, \dots w_n$, the language model specifies:

$$P(w_i \mid w_1, w_2, \dots, w_{i-1}).$$

For each position i, where:

$$\sum_{w \in V} P(w \mid w_1, w_2, \dots, w_{i-1}) = 1$$

This gives a valid probability distribution over the vocabulary V for every possible history.

Markov Assumption

Challenge: From data (text corpus), we cannot estimate reliably for long histories due to data sparsity.

Solution: Limit the context window!

Modern Large Language Models (LLMs) overcome this challenge using neural networks to learn abstract patterns and representations rather than memorizing exact sequences, enabling them to generalize to unseen contexts and handle histories of thousands of words through Transformer architectures with attention mechanisms.

Markov Assumption: The probability of a word depends only on a fixed number of previous words.

Consider a k-th order Markov Model (**order** refers to the number of previous states it conditions on):

$$P\left(w_i \mid w_1^{i-1}\right) \approx P\left(w_i \mid w_{i-k}^{i-1}\right)$$

This is the idea behind $\boxed{n\text{-gram}}$ models where $n = k + 1$.

n-gram Models

Definition. A **k-th order Markov Model** assumes that the probability of the current word depends only on the k most recent previous words, not the entire history.

$$P(w_i \mid w_1, w_2, \dots w_{i-1}) = P\left(w_i \mid w_1^{i-1}\right) \approx P\left(w_i \mid w_{i-k}^{i-1}\right)$$

where w_{i-k}^{i-1} denotes the sequence of k words from position $(i-k)$ to $(i-1)$.

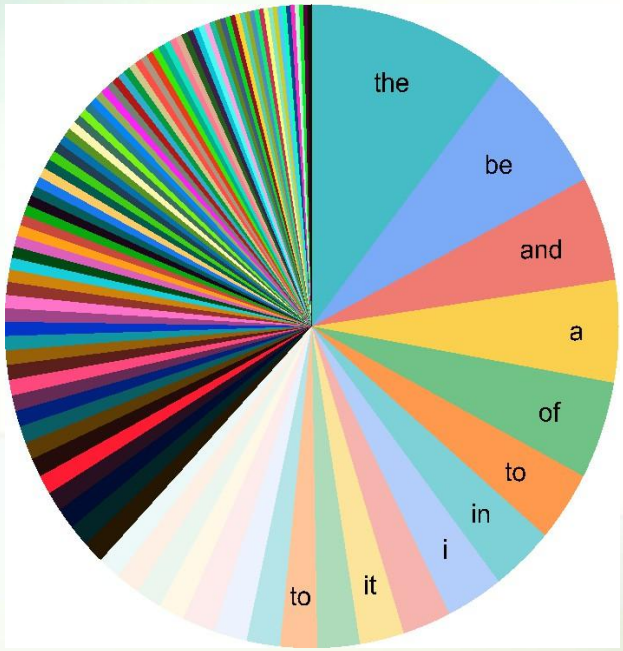
Markov Order	Conditions On	n-gram Name	Formula $P\left(w_i \mid w_1^{i-1}\right) \approx$	Notes
0th	0 previous words	Unigram	$P(w_i)$	Words chosen independently from a distribution
1st	1 previous words	Bigram	$P(w_i \mid w_{i-1})$	
2nd	2 previous words	Trigram	$P(w_i \mid w_{i-2}, w_{i-1})$	
k^{th}	k previous words	$(k+1)$-gram	$P\left(w_i \mid w_{i-k}^{i-1}\right)$	

Example: Unigram Model

Training: Count word frequencies in corpus

$$P(w_i) = \frac{\text{count}(w_i)}{N}$$

where N is the total number of words in corpus.



Sample probability distribution for a corpus.

Some generated sentences from a unigram model: Jurafsky Slides

- fifth, an, of, futures, the, an, incorporated, a, a, the, inflation, most, dollars, quarter, in, is, mass
- thrift, did, eighty, said, hard, 'm, july, bullish
- that, or, limited, the

- Individual words seem reasonable
- No coherence or grammatical structure
- Words appear with roughly correct frequencies

Example: Bigram Model

For Bigram models, the present depends on the immediate past.

Training: Count word pairs in the corpus

$$P(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}.$$

Some generated sentences from a bigram model: Jurafsky Slides

- texaco, rose, one, in, this, issue, is, pursuing, growth, in, a, boiler, house, said, mr., gurria, mexico, 's, motion, control, proposal, without, permission, from, five, hundred, fifty, five, yen
- outside, new, car, parking, lot, of, the, agreement, reached
- this, would, be, a, record, november

- Local coherence (pairs make sense)
- Some grammatical structure
- Still incoherent globally

Bigram Example

Example corpus

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I do not like green eggs and ham </s>
```

Pairwise Count Matrix

w_{i-1} \ w_i	<s>	I	am	Sam	do	not	like	green	eggs	and	ham	</s>	TOTAL
<s>	0	2	0	1	0	0	0	0	0	0	0	0	3
I	0	0	2	0	1	0	0	0	0	0	0	0	3
am	0	0	0	1	0	0	0	0	0	0	0	1	2
Sam	0	1	0	0	0	0	0	0	0	0	0	1	2
do	0	0	0	0	0	1	0	0	0	0	0	0	1
not	0	0	0	0	0	0	1	0	0	0	0	0	1
like	0	0	0	0	0	0	0	1	0	0	0	0	1
green	0	0	0	0	0	0	0	0	1	0	0	0	1
eggs	0	0	0	0	0	0	0	1	0	1	0	0	1
and	0	0	0	0	0	0	0	0	0	0	1	0	1
ham	0	0	0	0	0	0	0	0	0	0	0	1	1
</s>	0	0	0	0	0	0	0	0	0	0	0	0	0
TOTAL	0	3	2	2	1	1	1	1	1	1	1	3	17

$$P(\text{I} \mid \text{<s>}) = \frac{2}{3}$$

Example (continued)

For Bigrams: $P(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$

$P(\text{Sam} \mid \text{<s>}) = \frac{1}{3}$ $P(\text{am} \mid \text{I}) = \frac{2}{3}$ $P(\text{</s>} \mid \text{Sam}) = \frac{1}{2}$

$P(\text{<s>} \mid \text{Sam}) = 0$ $P(\text{do} \mid \text{I}) = \frac{1}{3}$ $P(\text{green} \mid \text{not}) = 0$

$P(\text{Sam} \mid \text{am}) = \frac{1}{2}$

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I do not like green eggs and ham </s>
```

Example: Trigram Model

Training: Count word triples in the corpus

$$P(w_i \mid w_{i-2}, w_{i-1}) = \frac{\text{count}(w_{i-2}, w_{i-1}, w_i)}{\text{count}(w_{i-2})}.$$

Some generated sentences from a trigram model:

- neural network that perfectly fits a finite dataset
- statistical learning cannot justify parameter counts from interpolation alone in the same way
- any first order theory with an infinite model also has a countable model compression knowledge distillation generalization theory the epistemology of deep learning and the criticism of the bigger is better doctrine

- Better local coherence
- More grammatical
- Still lacks long-range coherence

Question

Which of the following are bigrams from the sentence

"It is cold outside today"?

A. "It is" ✓

B. "cold today" ✗

C. "is cold" ✓

D. Both A and C ✓

Bigrams from this sentence:

- (It, is)
- (is, cold)
- (cold, outside)
- (outside, today)

In a bigram model, what assumption do we make?

A. All words are independent ✗

B. Each word depends on all previous words ✗

C. Each word depends only on the immediately previous word ✓

D. Words are chosen uniformly at random ✗

Maximum Likelihood Estimation (MLE)

Assume our training corpus is a *sample* from some *population* with unknown parameters θ. MLE finds the θ that maximizes the probability of observing this particular sample.

$$\theta^* = \arg\max_{\theta} P(\text{observed data} \mid \theta)$$

θ: The parameters we want to estimate (e.g., all bigram probabilities)

$P(\text{observed data} \mid \theta)$: $P(\text{observed data} \mid \theta)$: "If θ were the true parameters, how probable is the data we actually saw?

$\arg\max_{\theta}$: "The argument (value of θ) that maximizes..."

θ^*: The best parameter estimate.

For n-gram language models, the counting formula is the MLE estimator:

$$P\left(w_i \mid w_{i-k}^{i-1}\right) = \frac{\text{count}(w_{i-\text{k}}, \ldots, w_{i-1}, w_i)}{\text{count}(w_{i-\text{k}}, \ldots, w_{i-1})}$$

By the Law of Large Numbers, as training data size increases (toward ∞), MLE estimates converge to true probabilities.

Practical Issues: Log Probability Space

With large vocabularies (10,000+ words), individual word probabilities are small (e.g., P(word) ≈ 0.0001). For long sequences, we must multiply many such small probabilities together. This may result in ***underflow***.

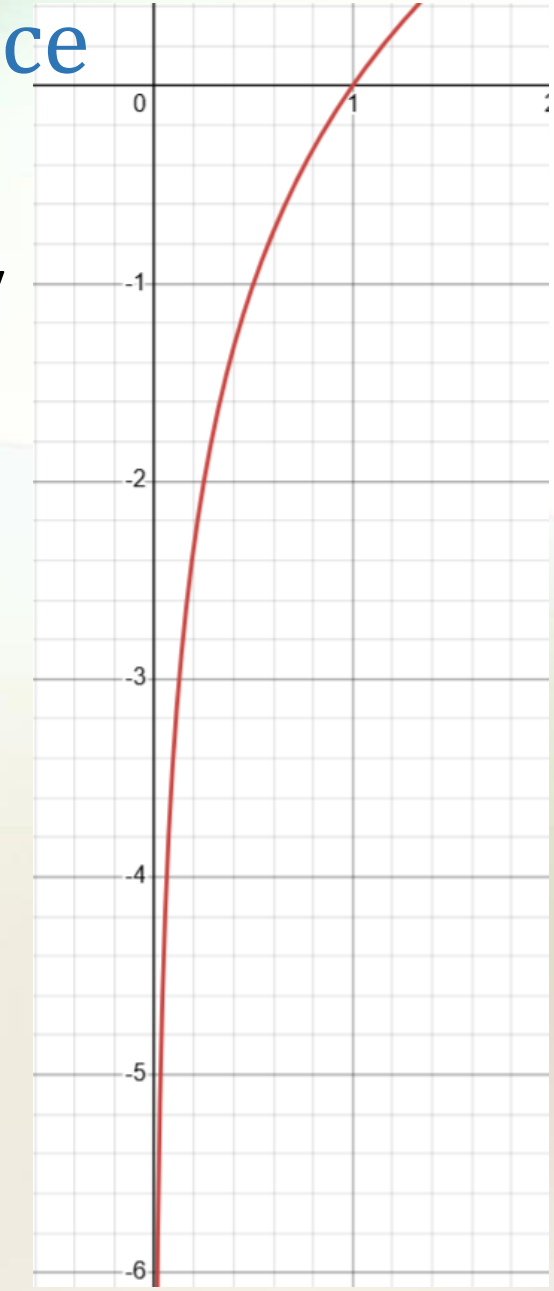
$$P(w_1^n) = \prod_{i=1}^{n} P\,(w_i | w_{i-1})$$

As a workaround we could work in log space:

$$\log P(w_1^n) = \sum_{i=1}^{n} \log P(w_i | w_{i-1})$$

This helps avoid underflow since addition is faster than multiplication.

It is also order preserving: $P(A) > P(B) \Leftrightarrow \log P(A) > \log P(B)$.



Zero Probability Problem

What if we encounter an n-gram never seen in training?

$$P(\text{word}_i|\text{word}_{i-1}) = \frac{\text{count}(\text{word}_{i-1}, \text{word}_i)}{\text{count}(\text{word}_{i-1})} = \frac{0}{\dots} = 0$$

The entire sentence would then have probability 0:

$$P(w_1^n) = \prod_{i=1}^{n} P\,(w_i|w_{i-1}) = \cdots \cdot 0 \cdot \cdots = 0$$

In the NLP setting, test data will always contain unseen n-grams. Thus, the model will become useless for unseen events.

Theoretically, real probabilities should be small, not zero.

Smoothing is a solution to this problem where we redistribute probability mass from the 'seen' data to the 'unseen'.

Smoothing: Add One

$$P(w_i \mid w_{i-k}^{i-1}) = \frac{\text{count}(w_{i-k}, \dots, w_{i-1}, w_i)}{\text{count}(w_{i-k}, \dots, w_{i-1})}$$

Unsmoothed

What if we pretend every n-gram has been observed at least one time?

Add-one Smoothing

$$P(w_i \mid w_{i-k}^{i-1}) = \frac{\text{count}(w_{i-k}, \dots, w_{i-1}, w_i) + 1}{\text{count}(w_{i-k}, \dots, w_{i-1}) + |V|}$$

Why is $|V|$ (where $|V|$ is vocabulary size) in the denominator?

$$\sum_{w \in V} P(w_i | w_{i-k}^{i-1}) = \sum_{w \in V} \frac{\text{count}(w_{i-k}, \dots, w_{i-1}, w_i) + 1}{\text{count}(w_{i-k}, \dots, w_{i-1}) + |V|}$$

$$= \frac{1}{\text{count}(w_{i-k}, \dots, w_{i-1}) + |V|} \sum_{w \in V} \text{count}(w_{i-k}, \dots, w_{i-1}, w_i) + 1$$ `# Constant denominator`

$$= \frac{1}{\text{count}(w_{i-k}, \dots, w_{i-1}) + |V|} \left[\sum_{w \in V} \text{count}(w_{i-k}, \dots, w_{i-1}, w_i) + \sum_{w \in V} 1 \right]$$

Observe, $\text{count}(w_{i-k}, \dots, w_{i-1}) = \sum_{w \in V} \text{count}(w_{i-k}, \dots, w_{i-1}, w_i)$. `# All words that follow` $w_{i-k}, \dots, w_{i-1}$

Also observe, $\sum_{w \in V} 1 = |V|$. So,

$$\sum_{w \in V} P(w_i | w_{i-k}^{i-1}) = 1$$

- Simple to implement
- No zero n-gram probabilities
- **Warning: Often reduces probability of seen events too much (over-smoothing)**

Generalized Smoothing: Add α

We can generalize smoothing by integrating a constant $0 < \boldsymbol{\alpha} < 1$ where $\boldsymbol{\alpha}$ represents how many imaginary observations we add to each n-gram (pseudo-counts).

General Smoothing

$$P\left(w_i \mid w_{i-\mathrm{k}}^{i-1}\right) = \frac{\mathrm{count}(w_{i-k}, \dots, w_{i-1}, w_i) + \boldsymbol{\alpha}}{\mathrm{count}(w_{i-k}, \dots, w_{i-1}) + \boldsymbol{\alpha}|V|}$$

Other benefits

- **More flexible than add-one**: α is a tunable hyperparameter
- **Less over-smoothing**: Reduces probability of seen events less dramatically
- **Better empirical performance**: $\boldsymbol{\alpha} < 1$ typically works better than $\boldsymbol{\alpha} = 1$.
- **Tunable on held-out data**: Optimize $\boldsymbol{\alpha}$ on a validation set for best performance
- **Controls bias-variance tradeoff**:
  - Smaller α → lower bias, higher variance;
  - larger α → higher bias, lower variance

Bias–variance tradeoff is the balance between underfitting (too simple, high bias) and overfitting (too complex, high variance) to minimize prediction error.

Smoothing Matters for Large n

Suppose $|V| = 20000$ and let k refer to a sequence of consecutive terms: k-grams.

How many k-grams are possible? $|V|^k$

Now suppose we have a corpus of size $N = 1{,}000{,}000$.

We can compute a sparsity ratio: $Sparsity = \frac{V^k}{N}$

What is the maximum number of distinct k-grams that would be observable in the corpus?

On the other of N

k-gram	V^k	Corpus (N)	Sparsity	Fraction Unseen
$k = 1$ (unigram)	20,000	1,000,000	0.02	0.02
$k = 2$ (bigram)	400M	1,000,000	400	> 0.997
$k = 3$ (trigram)	8T	1,000,000	8M	> 0.9999
$k = 4$ (4-gram)	160 quadrillion	1,000,000	160B	≈ 1.0

$$\lim_{k \to \infty} \frac{V^k}{N} = \infty$$

Since V^k grows exponentially while the corpus is fixed, larger k-grams are almost never observed. Smoothing is essential to handle the inevitable unseen sequences in test data.

Question

Smoothing is increasingly useful for n-grams when:

A. n gets larger ✓

B. n gets smaller

C. Always the same

D. n larger than 10

Larger n implies:

- more possible n-grams
- More sparsity in training data
- More zero counts

There is a greater need to handle unseen events with more zero counts.

Language Model Evaluation

Extrinsic Evaluation (End-to-End)

Approach: Embed model in application (translation, speech recognition) and measure task-specific performance (accuracy, error rate, quality scores)

Example: Compare language models by their impact on spelling correction accuracy

Intrinsic Evaluation (Standalone)

Method: Evaluate language model in isolation on held-out test data

Measure: ***Perplexity***—how well the model predicts test sequences

Example: Compute perplexity on test corpus, lower is better

Positives

- Directly meaningful - measures what we care about
- Task-relevant and realistic
- Gold standard for model selection

- Fast: no need to build full application
- Easy to compare: single numeric metric
- Reproducible: standard benchmark datasets

Negatives

- Expensive: requires full application pipeline
- Slow: each evaluation takes significant time
- Confounded: improvements could be from other system components, not the language model

- May not correlate with downstream performance
- Lower perplexity ≠ better application results
- Disconnected from real end-to-end goals

Entropy, Cross Entropy, and Perplexity

Measuring Uncertainty

How do we measure how ***uncertain*** ('surprised') we are by an outcome?

Examples

Flipping a fair coin

Extremely Uncertain (50/50)

Rolling a weighted die that almost always shows 6.

High Certainty

NLP Example: Sentiment analysis

"This movie was terrible!"

High certainty

"The film had some good moments but overall disappointing."

High uncertainty

"This is the best movie ever!"

High Certainty

We need a mathematical way to quantify this uncertainty.

Information Content

Information content refers to the number of bits required in an optimal binary encoding to represent an event.

Intuition

How many yes/no questions does it take to identify which event occurred?

When there are…

1 possibility?	0
2 possibilities?	1
4 possibilities?	2
8 possibilities?	3

More Abstractly

For n equally likely outcomes, the number of bits required is $\log_2 n$.

Also observe that the probability of each outcome is $p = \frac{1}{n}$.

If we combine these two observations, we have:

$$\log_2 n = \log_2 \frac{1}{p}$$

Definition. The ***information content*** (or self-information) of an event x with probability $p(x)$ is given by:

$$I(x) = -\log_2 p(x) = \log_2 \frac{1}{p(x)}$$

measured in bits.

Binary Entropy

$I(x) = -\log_2 p(x)$

Consider a biased coin with $P(Heads) = 0.8$ and $P(Tails) = 1 - P(Heads) = 0.2$.

In terms of information content of each outcome:

$$I(Heads) = -\log_2 0.8 \approx 0.32$$

$$I(Tails) = -\log_2 0.2 \approx 2.32$$

What is the average information per coin flip?

If we want to find the average information content per flip of the coin, we can compute the ***expected value***:

$$0.8 \cdot 0.32 + 0.2 \cdot 2.32 \quad = 0.72$$

Definition. For a binary random variable X with outcomes positive $\oplus$ and negative $\ominus$ over a collection S, the ***entropy*** is given by

$$Entropy(S) = -p_{\oplus} \log_2 p_{\oplus} - p_{\ominus} \log_2 p_{\ominus}$$

For simplicity, we define $0 \cdot \log_2 0 = 0$.

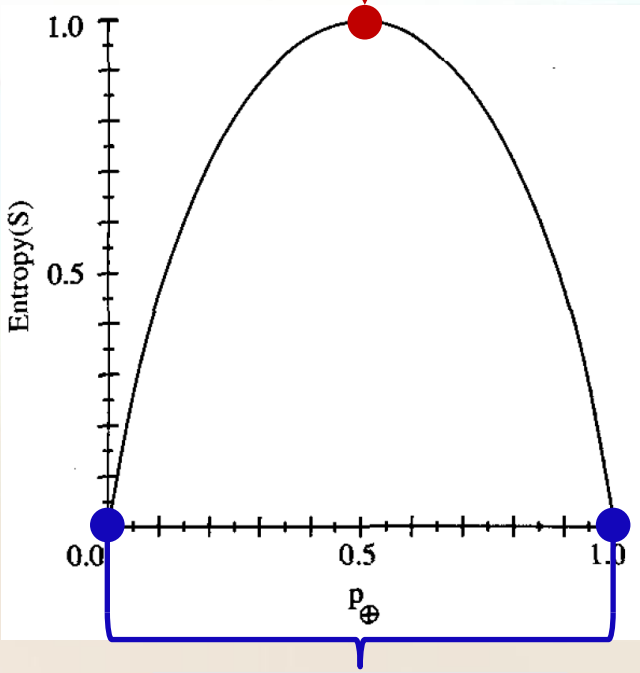
Binary Entropy

All observations are 50% one class and 50% the other.

Maximum uncertainty

$Entropy(S) = 1$

Entropy measures the average information content per trial.



Tom Mitchell, Machine Learning, page 57.

FIGURE 3.2
The entropy function relative to a boolean classification, as the proportion, $p_{\oplus}$, of positive examples varies between 0 and 1.

$Entropy(S) = 0$

Complete certainty

All observations in our data are the same classification.

Exercise

Estimate $Entropy(Heads = 0.9, Tails = 0.1)$

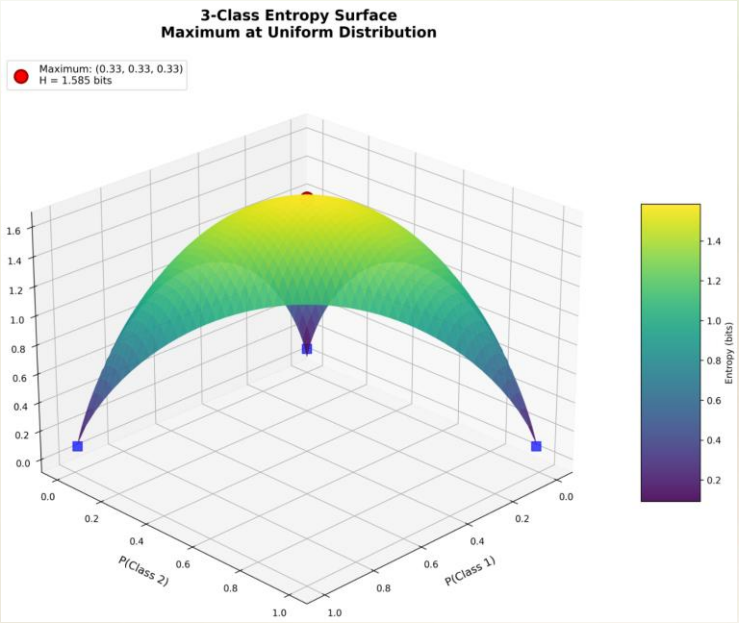
Extending Entropy to C Classes

Definition. For a random variable X with K possible outcomes $(c_1, c_2, \dots, c_K)$ over a collection S, the ***entropy*** is given by

$$Entropy(S) = -\sum_{i=1}^{K} p_{c_i} \log_2 p_{c_i}$$

where p_{c_i} is the proportion of examples in S with outcome c_i.

Consider the entropy of a collection with 3 classes visualized at the right.



All Class 1: $Entropy(S) = -(1\log_2 1 + 0\log_2 0 + 0\log_2 0) = 0$

Uniform: $Entropy(S) = -3 \cdot \left(\frac{1}{3}\log_2 \frac{1}{3}\right) = 1.585$

Model Comparison

Recall that a ***model*** is a system that assigns probabilities to possible outcomes.

Also recall that we discussed language models (e.g., n-grams) output a probability distribution $\boldsymbol{Q}$ over possible next words, but …

but the words that actually occur follow some true distribution $\mathbb{P}$.

$\boldsymbol{M}$

Predicted Distribution

$\mathbb{T}$

Ground Truth Labels

How well does $\boldsymbol{M}$ approximate $\mathbb{T}$?

Cross Entropy

$\mathbb{T}$ ***Entropy*** measures uncertainty when we know the true distribution. $Entropy(S) = -\sum_{i=1}^{K} p_{c_i} \log_2 p_{c_i}$

Definition. For two probability distributions $\mathbb{T}$ and $\boldsymbol{M}$ over the same set of outcomes $\{c_1, c_2, \dots, c_K\}$, the ***cross entropy*** of $\mathbb{T}$ with respect to $\boldsymbol{M}$ is given by

$$CrossEntropy(\mathbb{T}, \boldsymbol{M}) = -\sum_{i=1}^{K} p_{c_i} \log_2 q_{c_i}$$

where p_{c_i} is the true probability of outcome c_i and q_{c_i} is the predicted probability of outcome c_i.

This formula can be interpreted as:

- Events occur with true probabilities p_{c_i} according to $\mathbb{T}$
- A model $\boldsymbol{M}$ assigned probabilities q_{c_i} to each event
- When event c_i occurs, the information content is given by $\log_2 q_{c_i}$.

Cross entropy is thus the average information content across all events when using model $\boldsymbol{M}$ to predict events that actually occur according to distribution $\mathbb{T}$.

Example

Suppose we are building a POS tagger with 4 classes: {noun, verb, adjective, adverb}.

Consider the sentence: "The quick fox jumps over the lazy dog."

Our goal is to 'tag' the word *jumps*.

We know the word jumps can be represented using the tagging vector: $\mathbb{T}: [0, 1, 0, 0]$

Suppose our model predicts a tagging vector: $\boldsymbol{M}: [0.1, 0.7, 0.05, 0.15]$

$$CrossEntropy(\boldsymbol{M}, \mathbb{T}) = -[0 \cdot \log_2 0.1 + 1 \cdot \log_2 0.7 + 0 \cdot \log_2 0.05 + 0 \cdot \log_2 0.15]$$

$$\approx 0.51$$

Observe: lower cross entropy → better prediction

Question

Consider two events: $P(A) = \frac{1}{2}$ and $P(A) = \frac{1}{8}$. What are their information contents?

A. $I(A) = 2, I(B) = 8$

B. $I(A) = 1, I(B) = 3$ ✓

C. $I(A) = 3, I(B) = 1$

D. $I(A) = 0.5, I(B) = 0.125$

E. $I(A) = I(B)$

$$I(A) = -\log_2\left(\frac{1}{2}\right) = 1$$

$$I(B) = -\log_2\left(\frac{1}{8}\right) = 3$$

Suppose we are interested in modeling a fair coin flipping task. However, our model M finds $P(M) = [0.53, 0.47]$. What is the binary cross entropy?

A. 0.5

B. 0.72

C. 1.00 ✓

D. 1.50

$$CE(\mathbb{T}, \boldsymbol{M}) = -\frac{1}{2}\log_2 p(0.53) - \frac{1}{2}\log_2 p(0.47) \approx 1.00$$

If the ground truth itself is uncertain, even a perfect model will have “high” cross entropy.

Perplexity

Definition. For two probability distributions $\mathbb{T}$ and $\boldsymbol{M}$ over the same set of outcomes, the ***cross entropy*** of $\mathbb{T}$ with respect to $\boldsymbol{M}$ is given by

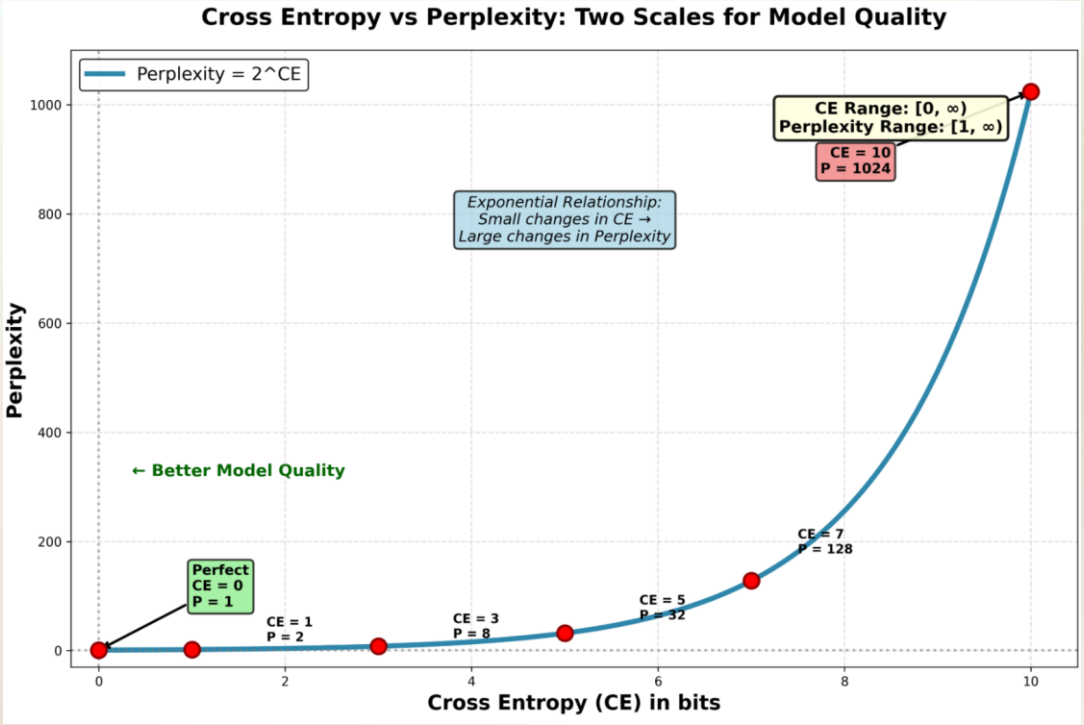
$$Perplexity(\mathbb{T}, \boldsymbol{M}) = 2^{CE(\mathbb{T},\boldsymbol{M})}$$

where $CE(\mathbb{T}, \boldsymbol{M}) = -\sum_{i=1}^{K} p_{c_i} \log_2 q_{c_i}$.

Perplexity is a transformation of the measurement into a more intuitive scale. It is widely used for evaluating language models in NLP.

Co-domains (ranges)

CE	$[0, \infty)$
$Perplexity$	$[1, \infty)$



Perplexity (for NLP)

Definition. Given a sequence of tokens $W = (w_1, w_2, \dots, w_n)$ and a language model that assigns conditional probabilities $P(w_i \mid w_1, \dots, w_{i-1})$, the ***perplexity*** of the model on this sequence is given by:

$$\boxed{Perplexity(W)} = 2^{CE(W)}$$

where the cross entropy $CE(W)$ is defined as:

$$\boxed{CE(W)} = -\frac{1}{n}\sum_{i=1}^{n} \log P(w_i | \mathrm{w}_1, \dots w_{i-1})$$

I thought we defined cross entropy and hence perplexity using two distributions? A ground truth $\mathbb{T}$ and model $\boldsymbol{M}$ for prediction.

Cross Entropy (for NLP)

$$CE(\mathbb{T}, \boldsymbol{M}) = -\sum_{i=1}^{K} p_{c_i} \log_2 q_{c_i}$$

$$CE(W) = -\frac{1}{n}\sum_{i=1}^{n} \log P(w_i | \mathrm{w}_1, \dots w_{i-1})$$

The sequence formulation is actually applying our cross entropy definition at each position: $w_1, \dots, w_n$.

The true distribution $\mathbb{T}_i$ is a one-hot distribution in which the actual next word w_i has probability 1, while all other words $(w_1, \dots, w_{i-1}, w_{i+1} \dots, w_n)$ have probability 0.

The model's predicted probability distribution over the vocabulary is $\boldsymbol{M}_i$. The cross entropy at position i is:

$$CE(\mathbb{T}_i, \boldsymbol{M}_i) = -\sum_{w \in \mathrm{vocab}}^{N} p_i(w) \log_2 q_i(w) = -1 \cdot \log_2 q_i(w_i) = -\log_2 P\left(w_i \mid w_1^{i-1}\right)$$

Then we average over all n positions in our sequence.

$$CE(W) = \frac{1}{n}\sum_{i=1}^{n} CE(\mathbb{T}_i, \boldsymbol{M}_i) = -\frac{1}{n}\sum_{i=1}^{n} \log P(w_i | \mathrm{w}_1, \dots w_{i-1})$$

Perplexity (for NLP)

Perplexity is essentially the *average uncertainty* the model has when choosing the next word.

Think of perplexity as a branching factor: the *effective number of choices* the model has at each step.

$perplexity = 10$

Perfect certainty; only one choice.

$perplexity = 50$

The model has 50 equally probable options for the next word.

$perplexity = 1000$

High uncertainty. Like having 1000 equally likely options for the next word.

Low perplexity → Model is confident:
assigns high probabilities to actual words in the sequence.

High perplexity → Model is "confused":
Assigns low probabilities to the correct words.

Question

Two language models are evaluated on the same test set:

Model A: Cross Entropy = 6 bits Model B: Cross Entropy = 4 bits

Which statement is true?

A. Model A has lower perplexity and is better

✓ B. Model B has lower perplexity and is better

C. Both models have the same perplexity

D. We cannot determine which is better without knowing the vocabulary size

Lower cross entropy → lower perplexity → better model.

What is the relationship between cross entropy $CE(\mathbb{T}, \boldsymbol{M})$ and the entropy $CE(\mathbb{T})$ of the true distribution?

A. $CE(\mathbb{T}, \boldsymbol{M})$ is always less than $CE(\mathbb{T})$

✓ *B.* $CE(\mathbb{T}, \boldsymbol{M})$ is always greater than or equal to $CE(\mathbb{T})$

C. $CE(\mathbb{T}, \boldsymbol{M})$ is always equal to $CE(\mathbb{T})$

D. There is no consistent relationship between them

$CE(\mathbb{T}, \boldsymbol{M}) \geq CE(\mathbb{T})$ always, with equality if and only if $\mathbb{T} = \boldsymbol{M}$ (the model perfectly matches the true distribution)

Perplexity: Example Results

Wall Street Journal corpus (40M words):

Model	Perplexity
Unigram	962
Bigram	170
Trigram	109

Jurafsky, D., & Martin, J. H. Speech and Language Processing (3rd ed. draft). Stanford University.

Some Observations:

Larger context → lower perplexity

- Conditioning on more context reduces entropy.
- More previous words enable a model to leverage syntactic patterns, semantic constraints, and words that frequently appear together (collocational preferences) for better predictions.

Model improvement does not scale linearly with context window size:

Trigram is 8.8 times better than unigram (not 3 times)

Diminishing returns for larger n

- Early context words resolve critical ambiguities; later words address subtler, rarer distinctions.
- Data sparsity worsens with larger n

Real systems often use 4-grams or 5-grams

Word Embeddings

Word Representations

Problem: How do we represent words to a computer?

Options	**Issue?**
Strings - “dog”, “cat”, “run”	Hard to compute with since there is no notion of similarity
Indices - dog = 1, cat = 2, run = 3, …	Easy to compute with Arbitrary: why is dog closer to cat than to run?
Embeddings - Vectors that capture meaning	Desired. Both computable and meaningful: similar words have similar vectors

For an NLP system to understand language, they need to understand, for example:

- “cheap” and “inexpensive” mean similar things
- “king” relates to “queen” like “man” relates to “woman”
- “excellent” is closer to “great” than to “terrible”

Desired Properties of Word Embeddings

Property	Description	Example
Semantic Similarity	Words with similar meanings → similar vectors	“happy” and “joyful” should be close
Capture Relationships	Vector arithmetic reflects word relationships	Paris - France + Italy ≈ Rome
Dense & Low-Dimensional	Typically 50-300 dimensions (not 100,000+)	Efficient to store and compute with
Syntactic Similarity	Words with similar grammar cluster together	Verbs near verbs, nouns near nouns
Measurable Distance	We can compute similarity between vectors	Cosine similarity enables applications

One-Hot Vectors

Representation

Each word is represented as a vector with a single 1 at its index position:

$$dog = [0,0,0,0,1,0,\dots,0]$$
$$cat = [0,0,1,0,0,0,\dots,0]$$

where the vector dimension is the vocabulary size $|V|$.

Properties

Property	Description
High-Dimensional	Vector size $|V|$ is typically between 10,000 - 100,000+
Sparse	Only one non-zero entry per vector
Orthogonal	All word vectors are orthogonal: $\mathbf{w}_i \cdot \mathbf{w}_j = 0$ for all $i \neq j$

Problems

Property	Description
No Semantic Similarity	All words are equally different: $\mathbf{w}_{dog} \cdot \mathbf{w}_{cat} = 0$ But we want *dog* and *cat* more similar than *dog* and *run*!
High Dimensionality	Computationally expensive, memory intensive
Cannot Capture relationships	No way to represent: `king:queen :: man:woman`

Solution

Learn dense, low-dimensional embeddings where similar words have similar vectors!

Distributional Semantics

"You shall know a word by the company it keeps" (Firth, 1957)

The Distributional Hypothesis: Words that occur in similar contexts tend to have similar meanings.

Similar contexts → Similar meanings

"dog", "cat", "puppy", "pet" all appear with:

"fur", "animal", "cute", "adopt"

Dog, *cat*, *puppy*, and *pet* should have similar vector representations

Different contexts → Different meanings

- "dog" appears with: "bark", "walk", "leash"
- "calculator" appears with: "compute", "display", "equation"

These words should have dissimilar vector representations

Distributional Semantics Goals

Consider these sentences with a missing word:

"I need to feed my ___"

"She adopted a new ___"

"The ___ was sleeping peacefully"

Words like $\{dog, cat, puppy, pet, hamster\}$ all fit these contexts.

Thus, all these words should have similar vectors!

Goal

Create vector representations where:

- Each word is a point in $\mathbb{R}^d$ with d typically between 50 and 300.
- Similarity between vectors reflects semantic similarity:
  - **Similar meanings** → high similarity / small 'distance'
  - **Different meanings** → low similarity / large 'distance'

$$d(dog, cat) < d(dog, calculator)$$

$$d(king, queen) < d(king, banana)$$

$$d(dog, puppy) < d(dog, cat) < d(dog, calculator)$$

$$d(pizza, algorithm) > d(exam, test)$$

Question

Consider a vocabulary with 10,000 words. Which statement about one-hot vectors is false?

A. Each word is represented by a 10,000-dimensional vector
B. The dot product of any two different word vectors equals zero
C. Each vector has exactly one non-zero entry
✓ D. Words with similar meanings have similar vector representations

One-hot vectors are orthogonal (all equally different), so they cannot capture semantic similarity.

According to the distributional hypothesis "you shall know a word by the company it keeps", which pair of words would you expect to have representations that should be *most* similar?

A. "king" and "throne" (often appear together in sentences)
✓ B. "doctor" and "nurse" (appear in similar contexts with similar surrounding words)
C. "run" and "running" (same root word)
D. "cat" and "mat" (rhyme with each other)

"run" and "running" share morphology, not contexts. They do not consistently appear in similar surrounding word environments ("run a company," "run away," "go for a run"; "running water," "running late," "running shoes"). Distributional models won't make them as similar.

The distributional hypothesis is about similar contexts (similar surrounding words). "Doctor" and "nurse" both appear with words like "patient", "hospital", "medical", etc.

Word Embeddings (Dense Vectors)

Represent each word as a **dense, low-dimensional vector** with real-valued entries:

$$dog = [0.13, 0.87, -0.23, 0.46, 0.87, -0.31]$$
$$cat = [0.07, 1.03, -0.43, -0.21, 1.11, -0.34]$$

Observe, these vectors are similar but not identical.

Property	One-Hot Vectors	Word Embeddings
Dimension	$\lvert V \rvert \in [10{,}000, 100{,}000+)$	$d \in [50{,}300]$
Density	Sparse (one 1, rest 0s)	Dense (all non-zero)
Semantic Similarity	None (all orthogonal)	Captures similarity
How Created	Predetermined by index	Learned from data

Definition: Word Embedding

Much smaller

Definition. Let $V = \{w_1, w_2, \dots, w_n\}$ be a finite ***vocabulary*** (set of words), where $|V| = n$ denotes the vocabulary size. Let $d \in \mathbb{N}$ be the ***embedding dimension***, with $d \boxed{\ll} |V|$. Let $\mathbb{R}^d$ denote the d-dimensional Euclidean vector space:

$$\mathbb{R}^d = \{x_1, x_2, \dots, x_d \mid x_i \in \mathbb{R} \text{ for all } i \in \{1, \dots, d\}\}.$$

A ***word embedding*** is a function $\phi: V \to \mathbb{R}^d$ that maps each word $w \in V$ to a d-dimensional vector:

$$\phi(w) = \mathbf{e}_w = [\mathrm{e}_{w,1}, \mathrm{e}_{w,2}, \dots, \mathrm{e}_{w,d}] \in \mathbb{R}^d \text{ where } e_{w,i} \in \mathbb{R} \text{ for all } i \in \{1, \dots, d\}.$$

We call $\mathbb{R}^d$ the ***embedding space*** and $\mathbf{e}_w$ the ***embedding vector*** (or simply ***embedding***) of word w.

A word embedding maps each word to a low-dimensional vector, where the location of each word in the embedding space reflects its meaning.

Word Embedding Similarity: Cosine

Definition. Let $V = \{w_1, w_2, \dots, w_n\}$ be a finite ***vocabulary*** (set) of words, where $|V| = n$ denotes the vocabulary size. Let $d \in \mathbb{N}$ be the ***embedding dimension***, with $d \ll |V|$. Let $\mathbb{R}^d$ denote the d-dimensional Euclidean vector space:

$$\mathbb{R}^d = \{x_1, x_2, \dots, x_d \mid x_i \in \mathbb{R} \text{ for all } i \in \{1, \dots, d\}\}.$$

A ***word embedding*** is a function $\phi: V \to \mathbb{R}^d$ that maps each word $w \in V$ to a d-dimensional vector:

$$\phi(w) = \mathbf{e}_w = [\mathrm{e}_{w,1}, \mathrm{e}_{w,2}, \dots, \mathrm{e}_{w,d}] \in \mathbb{R}^d \text{ where } e_{w,i} \in \mathbb{R} \text{ for all } i \in \{1, \dots, d\}.$$

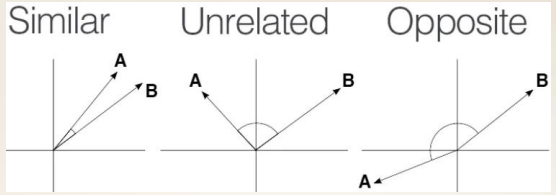
We call $\mathbb{R}^d$ the ***embedding space*** and $\mathbf{e}_w$ the ***embedding vector*** (or simply ***embedding***) of word w.

A word embedding maps each word to a low-dimensional vector, where the location of each word in the embedding space reflects its meaning.

Definition. Let $\mathbf{e}_{w_i}, \mathbf{e}_{w_j} \in \mathbb{R}^d$ be the embedding vectors for words $w_i, w_j \in V$. The ***cosine similarity*** between w_i and w_j is defined as:

$$\cos\theta = \frac{\mathbf{e}_{w_i} \cdot \mathbf{e}_{w_j}}{\|\mathbf{e}_{w_i}\| \cdot \|\mathbf{e}_{w_j}\|}$$

Cosine similarity measures the angle between two vectors; values near 1 indicating similar directions and values near 0 indicating orthogonal vectors.



Defining Context with Windows

The idea is that ***context*** equates to "words that appear nearby in a sentence".

The quick brown fox jumps over the lazy dog.

Consider the word $w_t = \text{fox}$; we will refer to this word as the ***center*** word.

Window Size	Context
2	{ brown, jumps }
3	{ quick, brown, jumps, over }
4	{ The, quick, brown, jumps, over, the }

Words close together are more related than distant words.

"quick" and "brown" tell us about "fox" more than "lazy" does.

A fixed window is also computationally tractable.

Similar Words Predict Similar Contexts

We aim to explore which contexts are predicted by different words.

dog { walked, in, park, the, barked, loudly, adopted, a, ... }

cat { petted, the, gently, meowed, softly, adopted, a, ... }

We observe that the resulting contexts are not mutually exclusive; some contexts overlap.

This overlap suggests that words sharing similar contexts tend to acquire similar representations and, by extension, similar meanings.

Building on this intuition, the ***Skip-gram*** approach seeks to learn word representations by optimizing them to predict surrounding contexts. Words that share similar contextual predictions will naturally converge toward similar vector representations.

Skip-Gram

Corpus: A sequence of words $w_1, w_2, \dots w_N$
Window size: a (typically $a \in [2,5]$)
Parameters: θ is all word vectors to be learned

'window size' is a misnomer since it refers to a radius

For each position t in the corpus sequence, we have

- w_t center word
- $w_{t-a}, \dots, w_{t-1}, w_{t+1}, \dots w_{t+a}$ context

Our objective is to maximize the probability of predicting context words from center words.

$$\mathcal{L}(\theta) = \prod_{t=1}^{N} \prod_{-a \le j \le a, j \ne 0} P\left(w_{t+j} \mid w_t; \theta\right)$$

Semicolon indicates θ as a learnable parameter (all the word vectors) we are conditioning on.

In practice, we maximize log-likelihood to avoid underflow:

$$\log \mathcal{L}(\theta) = \prod_{t=1}^{N} \prod_{-a \le j \le a, j \ne 0} \log P\left(w_{t+j} \mid w_t; \theta\right)$$

Intuitively, for every word in the corpus, we want high probability of predicting the words that actually appear around it.

Skip-Gram Captures Semantics

How do we model the probability of context word o given center word c?

Use two vectors per word:

Embedding vector when c is the center word: $\mathbf{v}_c \in \mathbb{R}^d$

Embedding vector when o is in the context: $\mathbf{u}_o \in \mathbb{R}^d$

Probability Model:

$$\boxed{P(o \mid c) = \frac{e^{\mathbf{u}_o \cdot \mathbf{v}_c}}{\sum_{w \in V} e^{\mathbf{u}_w \cdot \mathbf{v}_c}}}$$

Measures how compatible word o is as a context word for center word c

---

Sums the compatibility scores for **every possible word** in the vocabulary as a potential context word for c

This is a ***softmax*** function applied to our specific problem of word prediction.

Combining

$$\mathcal{L}(\theta) = \prod_{t=1}^{N} \prod_{-a \le j \le a, j \ne 0} P\left(w_{t+j} \mid w_t; \theta\right) \qquad P(o \mid c) = \frac{e^{\mathbf{u}_o \cdot \mathbf{v}_c}}{\sum_{w \in V} e^{\mathbf{u}_w \cdot \mathbf{v}_c}}$$

$\mathcal{L}(\theta)$ is the product of many individual probabilities computed using $P(o \mid c)$, one for each $(context, center)$ pair in the corpus.

Let the window radius be $a = 1$. Then,

$$\mathcal{L}(\theta) = \underbrace{\mathrm{P}\left(\mathbf{u}_{w_2} \mid \mathbf{v}_{w_1}\right)}_{\text{center at } w_1} \cdot \underbrace{\mathrm{P}\left(\mathbf{u}_{w_1} \mid \mathbf{v}_{w_2}\right) \cdot \mathrm{P}\left(\mathbf{u}_{w_3} \mid \mathbf{v}_{w_2}\right)}_{\text{center at } w_2} \cdot \underbrace{\mathrm{P}\left(\mathbf{u}_{w_2} \mid \mathbf{v}_{w_3}\right) \cdot \mathrm{P}\left(\mathbf{u}_{w_2} \mid \mathbf{v}_{w_3}\right)}_{\text{center at } w_3} \cdot \ldots$$

Example: Skip-Gram and θ

Suppose:

$V = [\text{dog}, \text{cat}, \text{run}]$

Embedding Dimension: $d = 3$

Parameter: θ contains all word vectors

Word	Center Vector $\mathbf{v}$	Context Vector $\mathbf{u}$
dog	$[0.2, 0.5, -0.1]$	$[0.3, -0.2, 0.4]$
cat	$[0.1, 0.6, -0.2]$	$[0.4, -0.1, 0.3]$
run	$[-0.5, 0.2, 0.8]$	$[-0.3, 0.1, 0.7]$

We thus have $|\theta| = 2 \cdot |V| \cdot d = 18$ total parameters.

These numbers are initially random, then adjusted through training to match the corpus.

These vectors are derived through training:

Gradient descent to iteratively adjust θ, maximizing the log-likelihood.

This increases the probability of context words that actually appear near each center word in the corpus.

Example: Skip-Gram and θ II

Word	Center Vector $\mathbf{v}$	Context Vector $\mathbf{u}$
dog	$[0.2, 0.5, -0.1]$	$[0.3, -0.2, 0.4]$
cat	$[0.1, 0.6, -0.2]$	$[0.4, -0.1, 0.3]$
run	$[-0.5, 0.2, 0.8]$	$[-0.3, 0.1, 0.7]$

What is the probability of "cat" appearing in the context of center word "dog"?

$$P(\mathbf{u}_{cat} \mid \mathbf{v}_{dog}) = \frac{e^{\mathbf{u}_{cat} \cdot \mathbf{v}_{dog}}}{\sum_{w \in V} e^{\mathbf{u}_{w} \cdot \mathbf{v}_{dog}}}$$

Candidate context:

$$e^{\mathbf{u}_{cat} \cdot \mathbf{v}_{dog}} = e^{[0.4,-0.1,0.3] \cdot [0.2,0.5,-0.1]} = e^{0.08-0.05-0.03} = 1$$

All possible word contexts:

$$\sum_{w \in V} e^{\mathbf{u}_{w} \cdot \mathbf{v}_{dog}} = e^{\mathbf{u}_{dog} \cdot \mathbf{v}_{dog}} + e^{\mathbf{u}_{cat} \cdot \mathbf{v}_{dog}} + e^{\mathbf{u}_{run} \cdot \mathbf{v}_{dog}}$$

$$= e^{0.06-0.10-0.04} + 1 + e^{-0.06+0.05-0.07}$$

$$= 0.9231 + 1 + 0.9231$$

$$= 2.8462$$

$$P(\mathbf{u}_{cat} \mid \mathbf{v}_{dog}) = \frac{1}{2.8462} \approx 0.3513$$

With current parameters θ, the model assigns 35% probability to "cat" appearing in "dog's" context.

Example: Skip-Gram and θ III

What is the relationship to the full objective $\mathcal{L}(\theta)$? $V = [\text{dog}, \text{cat}, \text{run}]$

$$\mathcal{L}(\theta) = \underbrace{\mathrm{P}(\mathbf{u}_{\text{cat}} \mid \mathbf{v}_{\text{dog}})}_{\text{center at dog}} \cdot \underbrace{\mathrm{P}(\mathbf{u}_{\text{dog}} \mid \mathbf{v}_{\text{cat}}) \cdot \mathrm{P}(\mathbf{u}_{\text{run}} \mid \mathbf{v}_{\text{cat}})}_{\text{center at cat}} \cdot \underbrace{\mathrm{P}(\mathbf{u}_{\text{cat}} \mid \mathbf{v}_{\text{run}})}_{\text{center at run}}$$

Recall, $\mathcal{L}(\theta)$ is the product of individual probabilities for every (context, center) pair in the corpus.

How Training Adjusts θ

Goal: Find θ that maximizes $\mathcal{L}(\theta)$ (or equivalently, $\log \mathcal{L}(\theta)$).

Process:

- Compute current $\mathcal{L}(\theta)$ using all corpus pairs
- If a pair like (walked, dog) appears in corpus but has low probability, adjust θ to
  - Increase $\mathbf{u}_{\text{walked}} \cdot \mathbf{v}_{\text{dog}}$
  - This makes $\mathbf{u}_{\text{walked}}$ and $\mathbf{v}_{\text{dog}}$ more aligned
- Repeat for millions of pairs, incrementally improving θ to increase $\mathcal{L}(\theta)$ (using gradient descent).

Words appearing in similar contexts (e.g., “dog” and “cat” both with “adopted”) end up with similar $\mathbf{v}$ vectors because learned embeddings capture semantic similarity.

After training, discard $\mathbf{u}_{\text{word}}$ or average with $\mathbf{v}_{\text{word}}$.

Word2Vec: Bringing It All Together

Word2Vec (Mikolov et al., 2013) is a practical algorithm for learning word embeddings that implements the Skip-gram objective with computational optimizations.

Efficient Estimation of Word Representations in Vector Space
by T Mikolov · 2013 · Cited by 50280 — Abstract:We propose two novel model architectu

Core Algorithm: Skip-Gram

- Given center word → predict context words
- Learns embeddings v_w that capture distributional semantics

Possible Optimizations (These make training feasible on large corpora (billions of words)**:**

- **Negative Sampling**: Instead of full softmax over V words, sample a few "negative" examples (~5-20) (words not in the context).
- **Subsampling**: Down-weight very frequent words ("the", "a", "is")

Word2Vec was revolutionary:

- Simple, efficient, and scalable
- Produced high-quality embeddings that capture semantic and syntactic relationships
- Made word embeddings accessible and practical for NLP applications
- Famous for capturing analogies: king - man + woman ≈ queen

Pre-trained embeddings:

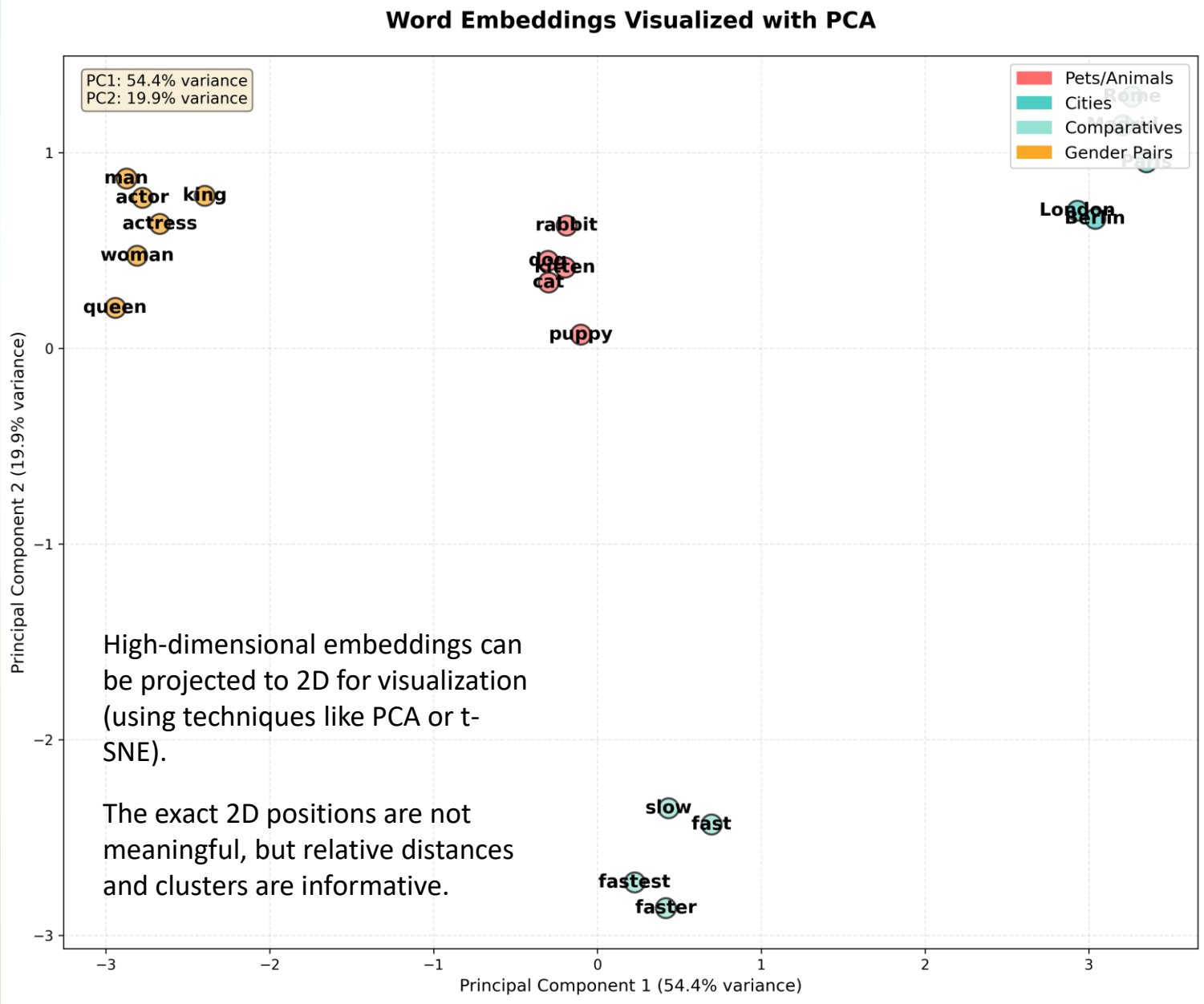
- Can download Word2Vec embeddings trained on Google News (3 billion words)
- Use immediately without training: plug-and-play for downstream tasks

Visualizing Word Embeddings

Similar words group together spatially (**semantic clustering**)

Multiple relationship types captured:

- **Semantic:** dog, cat, puppy, kitten (pets)
- **Geographic:** Paris, London, Rome, Berlin (cities)
- **Grammatical:** fast, faster, fastest (comparatives)
- **Gender:** king/queen, man/woman, actor/actress



High-dimensional embeddings can be projected to 2D for visualization (using techniques like PCA or t-SNE).

The exact 2D positions are not meaningful, but relative distances and clusters are informative.

Limitations of Simple Word Embeddings

1. No Context Sensitivity: Each word has a single fixed vector regardless of context

- "bank" (river) vs "bank" (financial institution) → same vector
- "bat" (animal) vs "bat' (baseball equipment) → same vector

2.Fixed Vocabulary: Cannot handle out-of-vocabulary (OOV) words

- Unseen words have no embedding: model trained on vocabulary V
- No representation for typos, rare words, new terms

3. No Compositionality: Unclear how to represent multi-word expressions

- How to get embedding for "New York" from "New" + "York"?
- Idioms like "kick the bucket" ≠ sum of parts

Modern models (like Transformers) address all three limitations simultaneously using subword tokenization, contextualized representations, and attention mechanisms.

Problem	Solution	How It Helps
No context	***Contextualized embeddings*** BERT, ELMo, GPT	Different vector for each word occurrence based on surrounding context
Fixed Vocabulary	***Subword tokenization*** (BPE, WordPiece, SentencePiece)	Break unknown words into known subword pieces
No Compositionality	***Contextualized Models*** with attention	Model interactions between words to represent phrases and sentences

Summary

Key Concepts

1. **Language Models:** Assign probabilities to word sequences
2. **N-grams:** Use Markov assumption to make estimation tractable
3. **Smoothing:** Handle unseen events (critical for larger n)
4. **Perplexity:** Intrinsic evaluation metric (lower is better)
5. **Word Embeddings:** Dense vectors capture semantic similarity
6. **Distributional Semantics:** "You shall know a word by the company it keeps"

Trade-offs

- Larger n → better modeling, but more sparsity
- Higher dimensions → more expressivity, but harder to train
- Simple methods (n-grams) still useful in many applications